

EEG Decoding: Common Spatial Pattern (CSP) + Linear Discriminant Analysis (LDA)

Introduction

The goal of this assignment is to familiarize you with classic EEG decoding pipelines. Here we use common spatial pattern (CSP) for feature extraction and linear discriminant analysis (LDA) for classification.

Dataset

The dataset provided with the assignment includes data from one subject downloaded from a publicly available dataset: <https://doi.org/10.6084/m9.figshare.21644429>.

You can also read more about the original study here:

doi: [10.3389/fnhum.2022.1019279](https://doi.org/10.3389/fnhum.2022.1019279)

Code

CSP_LDA_Assignment.py is the main Python file for the decoding task; **CSP.py/LDA.py** includes the functions for the CSP/LDA algorithm. You will need to complete the scripts by replacing all the ?s in the code, or you could write your own CSP and LDA algorithm if you prefer.

Part 1: Data Loading

Load the dataset provided with this assignment (**AssignmentData01.mat** and **AssignmentData02.mat**). This script was created for Python, but you can also load it into MATLAB. This is data from a recently published study that uses the same BCI setup as the one you will be using for the group project.

The assignment dataset should contain one variable: **BCI**, which contains four fields: **data**, **srate**, **TrialData**, and **metadata**

data: a list containing the raw EEG signals during each BCI trial. Each trial should contain 62 channels, one for each electrode, in the format [# of channels, # of samples]. The first 2000 data points are the 2 s of the intertrial interval before the current trial, the next 2000 data points are the 2 s of target presentation, and the last 1000 data points are the 1 s of the intertrial interval after the current trial. Thus, each trial is at least 5000 data points long and at max 11040 data points. In this assignment, we extract the first 2000 data points after the trial onset to ensure consistency in trial length.

srate: the sampling frequency of the data (1000 Hz).

TrialData: a list containing information about each trial, i.e., **prePostIntervention**, **taskParadigm**, **runNumber**, **trialNumber**, **targetNum**, **trialLength**, and **result**. **targetNum** represents the target that the subject attempted to move a cursor towards by performing motor imagery (1 is right in the LR task and up in the UD task, and 2 is left in the LR task and down in the UD task); **result** is the BCI cursor control result, noted as **hit** (the subject successfully moved the cursor towards and hit the target), **miss** (the subject moved the cursor in the wrong direction and hit the incorrect target), or **invalid** (the cursor ended up in the middle of the two potential targets). In this assignment, we only consider the **hit** trials as the successful trials.

A separate file **68channel.loc** specifies the location information of 68 channels in polar coordinates (polar angle and polar radius). Figure 1 shows the channel layout. Channel numbers [33, 43, 65, 66, 67, 68], or channel names ['M1', 'M2', 'VEO', 'HEO', 'EKG', 'EMG'], were not used for recording during the experiment. Feel free to experiment with different regions of interest to see how the results change.

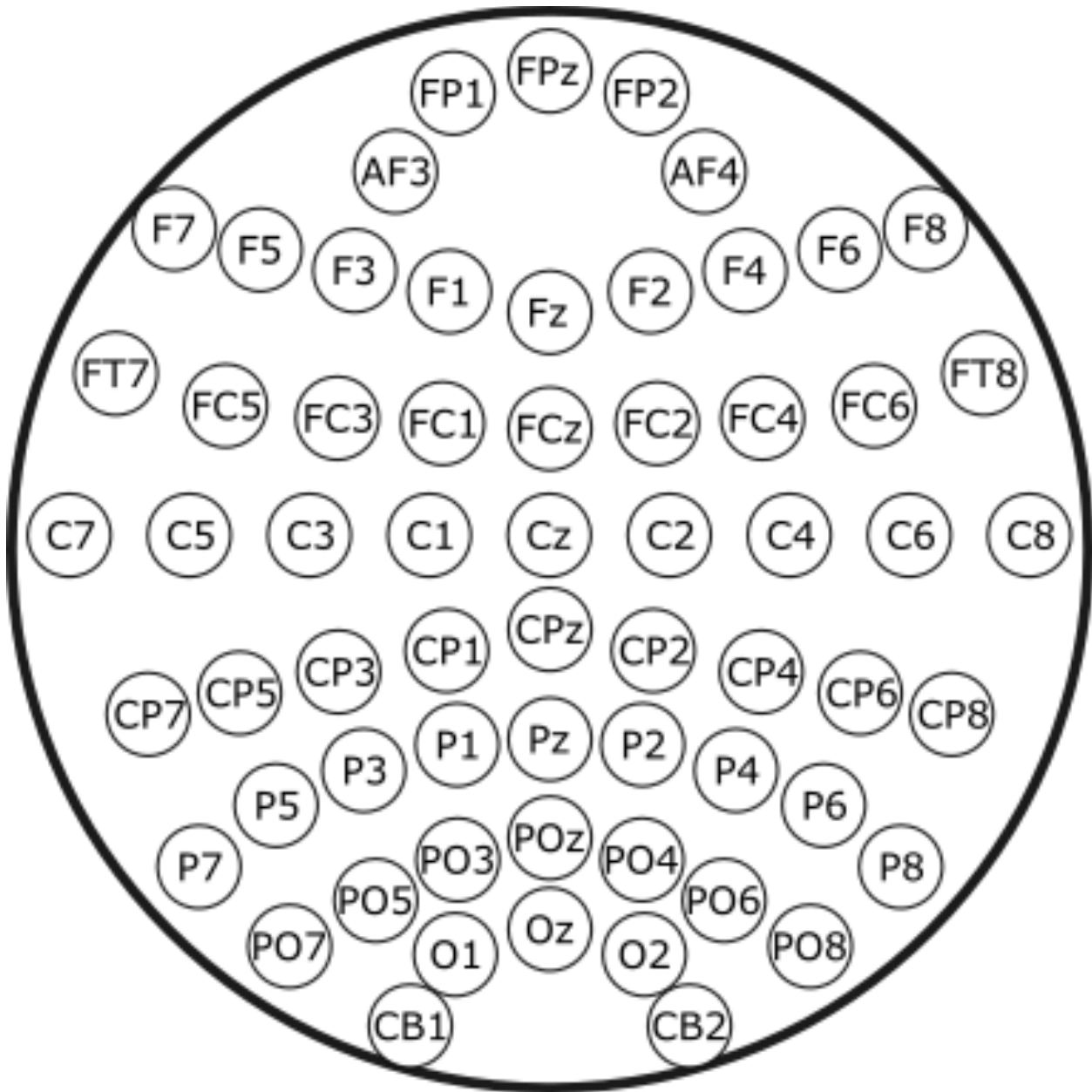


Figure 1: Neuroscan 64 Channel layout

Part 2: Data preprocessing

2a) Common average referencing is a simple re-referencing step that makes each EEG channel relative to the instantaneous scalp average. It helps remove reference bias and helps to remove shared noise, so spatial patterns are cleaner.

2b) Downsampling effectively reduces data size and computation while preserving the information you care about.

2c) Band-pass filtering keeps only the task-relevant frequency band and removes out-of-band noise/artifacts.

2d) (Optional) Selecting channels that are most task-relevant potentially makes it easier for the algorithm to focus on the most important features. However, too aggressive channel reduction may remove informative sensors and therefore negatively affect the performance. Other common EEG processing steps include temporal normalization and independent component analysis (ICA).

Part 3: Feature Extraction (CSP)

CSP learns a set of spatial filters, which are linear combinations of channels, from the preprocessed, labeled EEG signals. The goal is to maximize power for one class while minimizing it for the other, so that the filtered signal between classes is more separable.

3a) Class-wise covariance estimation.

For class k , concatenate all its epochs along time and compute the spatial covariance $C_k = (X_k X_k^T) / (T_k - 1)$, where $X_k \in \mathbb{R}^{d \times T_k}$ is the demeaned concatenation of epochs from class k . We optionally symmetrize: $C_k \leftarrow \frac{1}{2}(C_k + C_k^T)$.

Here we use a small ridge λ to ensure positive definiteness: $C_k \leftarrow C_k + \lambda \cdot \alpha_k \cdot I$, where α_k can be the average channel variance $\text{trace}(C_k)/d$.

3b) CSP objective and generalized eigenproblem.

CSP seeks spatial filters $w \in \mathbb{R}^d$ that maximize the variance ratio between classes:

$$J(w) = (w^T C_1 w) / (w^T C_2 w).$$

Maximizing $J(w)$ (or its symmetric counterpart) yields the generalized eigenproblem:

$$C_1 w = \lambda (C_1 + C_2) w, \text{ with } 0 < \lambda < 1.$$

λ equals the fraction of total variance explained by class 1, i.e., $w^T C_1 w = \lambda$ and $w^T C_2 w = 1 - \lambda$ under the constraint $w^T (C_1 + C_2) w = 1$.

3c) Component ordering.

In the two-class case, components are ranked in decreasing $|\lambda - 0.5|$ order. Large $|\lambda - 0.5|$ means the filter has very different power in the two classes; values near 0.5 indicate poor discrimination.

3d) Filters, patterns, and feature construction.

Stack the selected eigenvectors as columns to form $W \in \mathbb{R}^{d \times 2m}$, which are the $2m$ spatial filters we obtained. For an epoch $E \in \mathbb{R}^{d \times T}$, the CSP projections are $Z = W^T E \in \mathbb{R}^{2m \times T}$. For component j , the power is the time-average of squared amplitudes:

$$p_j = (1/T) \sum_{t=1}^T Z_{j,t}^2.$$

Features are typically log-powers: $f_j = \log(p_j + \varepsilon)$.

Part 4: Classification (LDA)

Once you have those CSP features, LDA draws the best single straight boundary (a hyperplane) to separate the two classes, assuming they have different means but a shared covariance in feature space:

$$x | y = k \sim \mathcal{N}(\mu_k, \Sigma), \quad k \in 1, 2$$

4a) Parameter estimates.

Sample means: $\hat{\mu}_k = (1/n_k) \sum_{i:y_i=k} x_i$.

Class covariances: $S_k = (1/(n_k - 1)) \sum_{i:y_i=k} (x_i - \hat{\mu}_k)(x_i - \hat{\mu}_k)^T$.

Pooled covariance (unbiased): $\hat{\Sigma} = ((n_1 - 1)S_1 + (n_2 - 1)S_2)/(n_1 + n_2 - 2)$.

For numerical stability, apply ridge: $\hat{\Sigma}_\lambda = \hat{\Sigma} + \lambda I$.

4b) Discriminant direction and intercept.

The Bayes/ Fisher-optimal linear direction is:

$$w = \hat{\Sigma}_\lambda^{-1}(\hat{\mu}_2 - \hat{\mu}_1)$$

With equal class priors $\pi_1 = \pi_2$, the intercept is:

$$b = -\frac{1}{2}(\hat{\mu}_1 + \hat{\mu}_2)^T w$$

For unequal priors, add $\log(\pi_2/\pi_1)$ to b .

4c) Decision rule.

Score each sample by $s(x) = w^T x + b$. The decision boundary $s(x) = 0$ is the hyperplane halfway between $\hat{\mu}_1$ and $\hat{\mu}_2$ in the Mahalanobis geometry induced by $\hat{\Sigma}_\lambda^{-1}$. With the b above, $s(\hat{\mu}_2) > 0$ and $s(\hat{\mu}_1) < 0$. The test samples are therefore classified by the sign of $s(x)$.

Report

a) Complete the scripts by replacing all the ?s in the code, or write your own CSP and LDA algorithm if you prefer. Report the results for the two data files provided for this assignment, including the accuracy and the feature visualization.

b) Based on the results you obtain, discuss the pros and cons of this approach compared with other decoding methods, like the linear classifier that is used for online control and other machine learning based approaches.

c) Explore different preprocessing approaches and discuss the effects of different preprocessing setups on the results. You could adjust the parameters of the existing steps or add/remove steps, like changing the temporal filtering frequency bands, changing the spatial filtering methods, changing the number of features, modifying the region of interest, ...

Please submit the completed code and write a report (~1 page) to report and discuss the results you obtain.